COAR: Computation-Aware Erasure-Coded Repair in Storage Clusters for Distributed Data Analytics

Mingqi Li[†], Yuchong Hu[†], Lin Wang[†], Chenxuan Yao[†], Patrick P. C. Lee[‡], Dan Feng[†]
[†]Huazhong University of Science and Technology, [‡]The Chinese University of Hong Kong

***Abstract*—Distributed data analytics clusters are widely used to process and store massive datasets. Erasure coding offers a cost-efficient mechanism for achieving data reliability, but incurs substantial bandwidth overhead for failure recovery. Most prior studies on erasure-coded repair focus on minimizing repair bandwidth. However, we observe that erasure-coded repair performance is affected not only by network bandwidth but also by the limited computational capacity under distributed data analytics workloads. We present COAR, a computation-aware erasure-coded repair scheme designed for storage clusters in distributed data analytics environments. COAR exploits the periodic computation patterns of analytics workloads to estimate repair computation capacity via the decoding throughput metric. It strategically assigns and schedules repair tasks by jointly considering computational and network resource states across nodes. Experiments conducted on Alibaba Cloud show that COAR reduces single-chunk, multi-chunk, and full-node repair time by up to 59.80%, 58.07% ,and 59.47%, respectively, compared with existing repair schemes under diverse analytics workloads.**

I. Introduction

The exponential growth of data volumes and computational demands has driven the widespread adoption of distributed data analytics [42]–[44]. To manage massive workloads, existing data analytics systems (e.g., Apache Spark [41]) rely on backend storage clusters (e.g., HDFS [40]) to persist petabyte-scale datasets and intermediate results [4], [34], [37], [45]. However, node failures in large-scale clusters can lead to data loss and service disruptions, making fault tolerance a critical requirement. *Replication* is the traditional approach to reliability by maintaining multiple data copies across nodes, but petabyte-scale storage makes replication prohibitively costly [4], [34].

Erasure coding offers a more storage-efficient alternative and has been widely adopted in modern storage clusters (e.g., Google Colossus [8], Microsoft Azure [14], and Facebook f4 [27]). It encodes k data chunks into m parity chunks to form a *stripe* in a way that allows reconstruction of any lost chunk from any k surviving ones, achieving similar reliability with far less redundancy [27], [38]. Despite its efficiency, erasure coding introduces a repair penalty: recovering one lost chunk requires fetching k chunks, amplifying bandwidth by $k\times$. Prior studies [2], [10], [13], [24], [26] have focused mainly on minimizing repair bandwidth, assuming that network transfer is the performance bottleneck.

The corresponding author is Yuchong Hu. This work was supported by the National Natural Science Foundation of China (No. 62272185, No. U25A20423), the Key Laboratory of Information Storage System, Ministry of Education of China, and Research Grants Council of Hong Kong (AoE/P-404/18).

However, our measurements in distributed analytics clusters reveal a different reality. The performance of erasure-coded repair is often limited by computation rather than bandwidth (Finding #1 in §III-A). In analytics workloads, CPUs are heavily utilized, while network resources are relatively abundant (Finding #2 in §III-A). This motivates a paradigm shift toward *computation-aware* erasure-coded repair that jointly considers computational and network resources. Nevertheless, achieving computation-aware repair poses a key challenge: *quantifying computational capacity with a bandwidth-like metric usable for joint optimization* (§III-C), which is non-trivial in practice due to the need for accurate bandwidth-like metric quantification and effective short-term profiling that captures computational availability and reflects repair-oriented computational capacity.

We address this challenge with COAR, a COmputation-Aware Erasure-Coded Repair mechanism that leverages the periodic computation patterns of analytics workloads to estimate repair performance via a periodicity-based approach, thereby enabling efficient repair task allocation and scheduling. Our contributions include:

- We measure and identify the interplay between computational and network resources for repairs in storage clusters for distributed data analytics (§III).
- We characterize the periodicity of computations under analytics workloads and model repair performance by jointly considering both computational and network resources (§IV).
- We design COAR, which accelerates single-chunk, multi-chunk, and full-node recovery through adaptive task allocation and scheduling (§V).
- We prototype and open-source COAR at https://github.com/YuchongHu/COAR. Experiments on Alibaba Cloud [1] show that COAR reduces single-chunk, multi-chunk and full-node repair time by up to 59.80%, 58.07%, and 59.47% compared to existing repair schemes, respectively, across diverse analytics workloads (§VI).

II. Background and Related Work

A. Distributed Data Analytics

Distributed data analytics has been widely used to process massive datasets [45], and various frameworks have been proposed. For example, Apache Spark [41] decomposes an analytics job into multiple *stages*. Operations without shuffle dependencies (e.g., map, filter) are grouped into the same stage, while shuffle operations trigger data exchanges between stages.

Distributed data analytics systems depend on the underlying storage layer to manage petabyte-scale datasets and intermediate computation results across stages [4], [19], [23], [45]. Spark supports multiple deployment paradigms, most notably *co-located* [36] and *disaggregated* [33] storage architectures. In disaggregated architectures, compute and storage are separated to improve resource isolation and scalability. In contrast, co-located architectures run Spark executors directly on storage nodes (e.g., HDFS DataNodes [40]), leveraging data locality to minimize cross-node data transfers and improve performance for compute-intensive workloads [39]. In this paper, we focus on the storage layer under co-located clusters.

The above co-located storage clusters (e.g., HDFS [40]) often employ fault tolerance mechanisms, primarily replication and erasure coding (§II-B), to ensure reliability against physical data corruption or storage node failures [27], [45]. This paper focuses on *data reliability* within storage clusters for distributed data analytics. Note that distributed data analytics also has fault-tolerance mechanisms (e.g., Resilient Distributed Datasets (RDDs) [49]) to ensure *computational reliability* and maintain the continuity of analytics jobs. In this paper, we focus on data reliability instead of computational reliability, and our study is orthogonal to existing research on computational reliability.

Distributed data analytics often supports a variety of workloads. We focus on three representative categories [36], each supported by Spark's unified analytics libraries:

- **Distributed Machine Learning (MLlib [43]):** It implements iterative algorithms for training, classification, and clustering, such as K-means clustering and logistic regression over large datasets, which are typically partitioned and distributed across nodes. In each iteration, nodes perform compute-intensive operations (e.g., gradient computation) on local partitions, followed by parameter synchronization across nodes.
- **SQL and Structured Data Processing (Spark SQL [44]):** It handles large-scale structured datasets as shards through complex extraction, transformation, and loading (ETL) workflows and interactive querying. Complex SQL queries are decomposed into multiple stages, with nodes executing local computations (e.g., filtering, aggregation). Shuffling is required to redistribute intermediate data (e.g., GROUP BY, JOIN) between stages.
- **Graph Processing (GraphX [42]):** It analyzes the structure of complex networks, such as PageRank and connected-component detection, performs computations (e.g., PageRank value updates) on local subgraphs, and exchanges vertex states.

In this paper, we will analyze the above representative data analytics workloads to show (i) how they affect storage performance (§III) and (ii) their computation patterns (§IV).

B. Erasure Coding

Given the high storage overhead of replication, we focus on storing data with *erasure coding* for fault tolerance. Erasure coding is characterized by two parameters, k and m, which define its fault tolerance and storage overhead. It encodes k fixed-size data chunks into m parity chunks, each computed as a linear combination of the k data chunks using Galois Field arithmetic [35]. The resulting $k+m$ chunks form a *stripe*, which is distributed across $k+m$ nodes to allow data recovery from up to m simultaneous node failures.

Among various erasure codes, Reed–Solomon (RS) codes [35] are the most widely deployed in production systems. Let $\mathrm{RS}(k,m)$ be the RS code with parameters k and m. RS codes are *maximum distance separable* (MDS), meaning that in $\mathrm{RS}(k,m)$, any k of the $k+m$ chunks suffice to reconstruct the original data. When a chunk D' fails, the repair process retrieves k surviving chunks of the same stripe ($\{D_1, D_2, \ldots, D_k\}$) and reconstructs D' as $D' = \sum_{i=1}^{k} \alpha_i D_i$, where α_i ($1 \le i \le k$) denotes the decoding coefficient of D_i to repair D'.

RS-coded repair incurs high bandwidth overhead, as $\mathrm{RS}(k,m)$ transmits k chunks for decoding computations. High repair penalty implies system performance degradation since serving requests for failed nodes depends on responses from k survivors. Also, prolonged repair durations extend the degraded state, reducing system reliability. Thus, extensive studies focus on accelerating erasure-coded repair, with the primary objective of reducing transmission time under the assumption that bandwidth dominates repair cost. We review four representative approaches as follows.

- **Conventional Repair (CR):** It is the baseline where surviving chunks are transmitted to a single node for decoding.
- **Partial Parallel Repair (PPR) [26]:** It divides reconstruction into smaller sub-tasks executed across multiple nodes to reduce network load.
- **Repair Pipelining (RP) [24]:** It schedules repair operations at the sub-chunk granularity across nodes in a pipelined fashion to reduce repair time.
- **ChameleonEC [2]:** It dynamically tunes the repair plan to minimize bandwidth contention with foreground traffic and better saturate unoccupied bandwidth.

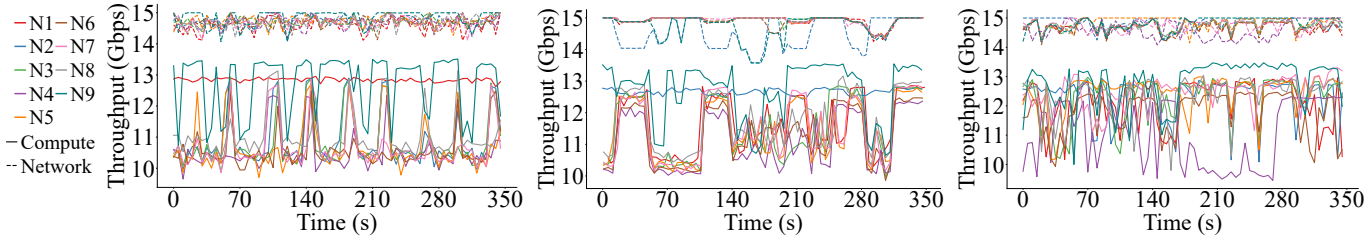
The above repair approaches often focus on network resource optimizations, but do not address the impact of computational resources. In §III, we provide a comprehensive study to motivate the importance of addressing computational overhead in erasure-coded repair under distributed data analytics.

Erasure coding has also been extensively studied in the context of data analytics clusters. Degraded-first scheduling [23] improves MapReduce performance under failure by launching degraded tasks earlier to utilize idle network bandwidth. Carousel codes [20] and Galloper codes [21] enhance parallelism, enabling faster data access and improving job performance. EC-Shuffle [48] further reduces shuffle traffic and computation overhead in Spark via erasure-coding-based shuffle optimization. Darrous *et al.* [4] evaluate the performance of data-intensive applications using replication versus erasure coding. However, these studies do not address the interplay of computational and network resources in erasure-coded repair.

III. Motivation

A. Findings

We examine how distributed data analytics workloads affect erasure-coded repair and the underlying resource utilization



(a) K-means clustering **(b)** OLAP queries **(c)** PageRank

Figure 1: Computation and network throughput under data analytics workloads.

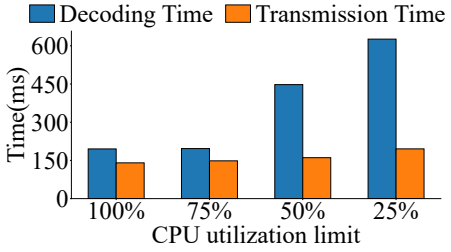


Figure 2: Decoding and transmission time in RS(4,2) repair by CR under different CPU utilization limits.

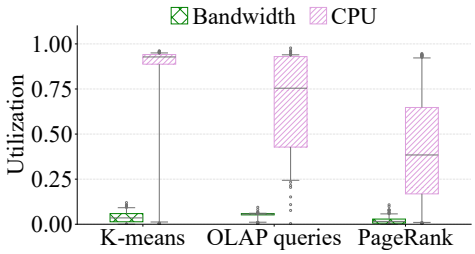


Figure 3: CPU and network utilization under data analytics workloads. CPU utilization is the total fraction of non-idle CPU milliseconds while the task is running, divided by the machine's total cores. Network utilization is the bandwidth usage divided by the machine's total bandwidth. Boxes depict 25th, 50th, and 75th percentiles; whiskers depict 5th and 95th percentiles.

via measurement. We perform measurements on a 10-node cluster comprising one master and nine worker nodes. Each worker hosts an HDFS DataNode and a Spark executor (see §VI-A), where Spark is used for distributed data analytics, while HDFS serves as the storage backend. We encode data with RS(6,3). We run three representative workloads: K-means clustering, PageRank, and OLAP queries (§II-A). To identify the limiting factors of repair performance, we measure two key metrics for repair: (i) *available network throughput*, defined as the minimum of a node's residual upload and download bandwidths that dictates the network bottleneck, and (ii) *available computation throughput*, defined as the real-time decoding rate and calculated as the ratio of decoded data size to decoding time. Our measurements yield two key findings.

Finding #1: *Decoding computation is the bottleneck during repair under analytics workloads.* As shown in Fig. 1, across all measured workloads, the available computation throughput is consistently lower than network throughput. This indicates that the compute-intensive analytics jobs significantly degrade decoding computation performance, challenging the traditional assumption that network transmission dominates repair time. To further investigate the sensitivity of repair performance to computational resources, we conduct a controlled micro-benchmark. We use the `cpulimit` command to restrict the maximum CPU usage of the repair process and measure the decoding and transmission time for RS(4,2) repair using CR. As shown in Fig. 2, the results reveal that as the CPU utilization limit decreases from 100% to 25%, the increase in decoding time is more pronounced than transmission time. This demonstrates that decoding operations are more sensitive to computational resources, making them prone to becoming the bottleneck when computational resources are scarce.

Finding #2: *CPU resources are scarcer than network resources in analytics clusters.* To understand the causes of the observed computation bottleneck in Finding #1, we further analyze CPU and bandwidth utilizations under different data analytics workloads. Fig. 3 shows that CPU utilization is consistently higher than bandwidth utilization across three workloads; this also conforms to the observation made in prior work [29]. This demonstrates that in compute-intensive data analytics environments, limited computational capacity, rather than bandwidth, is the primary constraint on erasure-coded repair performance.

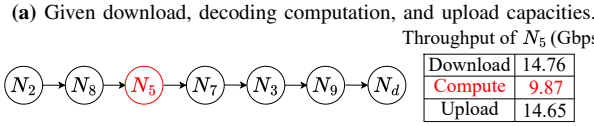
B. Motivating Example

In distributed data analytics clusters, the dominant bottleneck of erasure-coded repair shifts from data transmission to decoding computation (Finding #1), primarily because analytics workloads consume substantial CPU resources (Finding #2). However, existing erasure-coded repair strategies focus solely on network optimization (§II-B), neglecting computation. As a result, they often fail to minimize repair time in compute-intensive environments where bandwidth is abundant but computational capacity is constrained. This motivates a shift toward *computation-aware repair*, which *jointly* considers both computational and network resources.

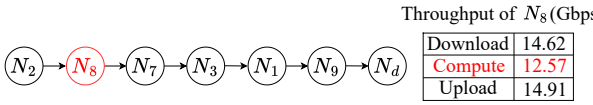
Fig. 4 illustrates this motivation under the K-means clustering workload. Consider an RS(6,3) stripe distributed across nine nodes (N_1 to N_9), where the chunk size is 64 MB. A repair is triggered when a newcomer N_d requests a chunk previously stored on the failed node, say N_6. Fig. 4(a) profiles each surviving node's upload, download, and computation throughput, reflecting its capability to transmit, receive, and decode data, respectively. These throughput profiles are captured from a specific time step of the K-means clustering in Fig. 1(a).

Throughput (Gbps)	N_1	N_2	N_3	N_4	N_5	N_7	N_8	N_9	N_d
Download	15.00	14.56	14.81	14.76	14.76	14.80	14.62	14.96	14.88
Compute	12.88	9.97	12.88	9.65	9.87	12.86	12.57	13.38	/
Upload	14.29	14.85	14.85	14.53	14.65	14.91	14.91	15.00	/

(a) Given download, decoding computation, and upload capacities.



(b) ChameleonEC.



(c) Computation-aware repair (our idea).

Figure 4: Comparison of ChameleonEC and computation-aware repair (our idea) under given capacities download, upload, and decoding computation capacities of different nodes.

We compare the state-of-the-art repair method (i.e., ChameleonEC [2], see §II-B) with our computation-aware idea that jointly considers both computational and network resources, as shown in Fig. 4(b) and (c), respectively. First, ChameleonEC neglects computational resources, causing decoding operations to become the bottleneck, thereby slowing down the whole repair process: ChameleonEC constructs the repair pipeline by selecting the nodes N_2, N_8, N_5, N_7, N_3, and N_9 in Fig. 4(b), solely based on their better network (download/upload) throughput. However, among them, N_5 has low computation throughput (i.e., 9.87 Gbps), stalling the repair pipeline.

In contrast, our computation-aware idea selects the nodes N_2, N_8, N_7, N_3, N_1, and N_9 in Fig. 4(c) to construct the repair pipeline, since they have both decent network and computation throughput, such that the slowest node N_8 has higher computation throughput than N_5 (12.57 Gbps vs. 9.87 Gbps). This example highlights the need to coordinate computational and network resources for erasure-coded repair, particularly under compute-intensive data analytics workloads.

C. Challenges

Enabling computation-aware repair requires jointly considering computation and network resources, yet *accurately quantifying computational resources* is challenging. The reason is that (i) unlike directly measurable network bandwidth, computational capacity is an implicit property influenced by multiple hardware factors (e.g., CPU cores and memory bandwidth [5], [9]); (ii) foreground data analytics workloads contend for computational resources and incur interference, making computation throughput hard to capture accurately.

A straightforward method to address the challenge is snapshot-based monitoring (e.g., heartbeats [46]), which uses instantaneous CPU utilization as the estimation metric, but often fails to reflect imminent transitions between compute-intensive and idle states [29]. Another method is to leverage historical statistics, which can be robust against transient fluctuations [31], yet it requires comprehensive and long-term profiling over entire job lifecycles.

Consequently, the fundamental challenges lie in developing an *accurate and effective estimation* mechanism that (i) quantifies computational capacity with an accurate metric comparable to network bandwidth (Challenge #1), and (ii) uses short-term profiling which effectively reflects the computational capacity for repair (Challenge #2). These two challenges will be addressed in §IV.

IV. Analysis and Model

In this section, we characterize the computation pattern of distributed data analytics workloads (§IV-A), estimate the decoding computational capacity based on the computation patterns (§IV-B), and model the computation-aware repair process (§IV-C).

A. Analyzing Distributed Data Analytics Workloads

To analyze the computation pattern of distributed data analytics, we conduct a measurement study on a 10-node Spark cluster under three workloads [28], [36]. Each of these workloads corresponds to the representative categories introduced in §II-A: (i) K-means clustering (for distributed machine learning), (ii) OLAP queries (for SQL analytics), and (iii) PageRank (for graph computing). The experimental configuration is detailed in §VI-B. By monitoring CPU utilization on each node, the results reveal that distributed data analytics workloads exhibit a *periodic* computation pattern, as shown in Fig. 5. For example, in Fig. 5(a), the selected observed nodes (N_1, N_5 and N_9) exhibit four periodic iterations of CPU utilization.

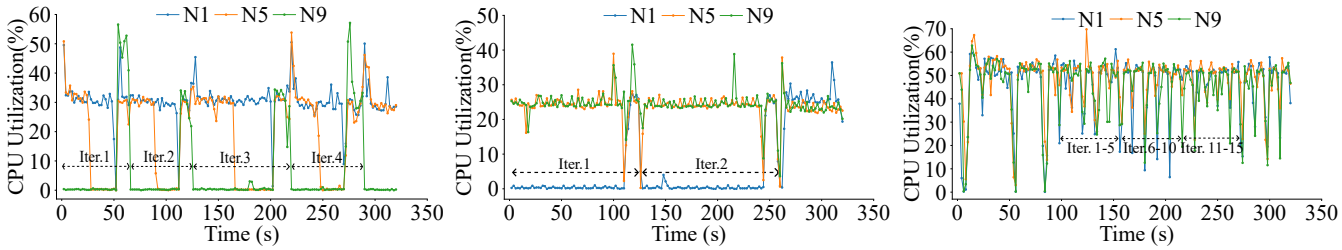
This periodic computation pattern stems from two inherent execution characteristics of distributed data analytics. First, *iterative execution*. Analytics jobs often repeatedly process massive datasets over multiple iterations (e.g., epochs in ML, supersteps in graph processing). Each iteration executes the same set of parallel computation and cross-node coordination operations [47]. Second, *staged execution*. Analytics workloads are composed of multiple stages in a Directed Acyclic Graph (DAG) structure [49], which partitions iterations into compute-intensive and compute-idle phases.

Therefore, the periodic computation pattern reveals that the variability of computational resources follows a *predictable* cycle. This property enables us to estimate the decoding throughput by leveraging short-term profiles within one periodic iteration, thus addressing Challenge #2 in §III-C.

B. Estimating Computational Capacity

Based on the periodic computation pattern analyzed above, we propose a *periodicity-based estimation* mechanism to address Challenge #1 in §III-C, including selecting and profiling the computation metric, specified as follows.

Decoding throughput metric selection. To estimate computational capacity as an accurate metric comparable to network bandwidth, we choose the *decoding throughput* (denoted by P), defined as the volume of data decoded per unit time. The reason is that (i) both decoding throughput and network bandwidth belong to the rate dimension (bits per second), and (ii) decoding



(a) K-means clustering **(b)** OLAP queries **(c)** PageRank

Figure 5: CPU utilization of some representative nodes under distributed data analytics workloads.

operations are more sensitive to computational resources and prone to becoming bottlenecks (see Fig. 2).

Periodicity-based profiling. We first maintain decoding profiles collected from historical decoding operations for each node in the cluster. Each profile is recorded as a tuple $\{\phi_j, U_j^{cpu}, B_j^{mem}, P_j\}$, representing the j-th decoding operation. Specifically, ϕ_j represents the time offset relative to the start of the respective iteration when the operation occurred. It allows us to align sparse decoding operations from multiple iterations into a unified iteration to construct a complete profile of decoding throughput. U_j^{cpu} and B_j^{mem} are the CPU utilization and memory bandwidth usage measured at the onset of this operation, respectively. P_j is the actual decoding throughput derived from the decoding operation (calculated as $\frac{S\times k}{T}$ where chunk size is S, chunk number is k, and decoding time is T).

Next, we employ these profiles to fit a lightweight XGBoost model [3] (denoted by $\mathcal{M}$) to predict the decoding throughput. We choose XGBoost because decoding throughput is jointly affected by the workload phase, CPU utilization, and memory bandwidth usage. Their effects are often non-linear under foreground analytics workloads. Note that while simple estimators (e.g., Exponential Weighted Moving Average) are computationally cheaper, they often capture recent trends and fail to anticipate the resource transitions of analytics workloads. As a result, they may react slowly to transitions between compute-intensive and compute-idle stages. In contrast, XGBoost captures non-linear dynamics from sparse samples with low inference latency. We evaluate its effectiveness and inference overhead in §VI-C to demonstrate that the extra complexity is small compared with repair execution time.

Finally, when a new repair task arrives, $\mathcal{M}$ captures the node's real-time state metrics, including the current time offset ϕ, CPU utilization U^{cpu}, and memory bandwidth B^{mem}. Then, we derive the estimated decoding throughput as:

$$\hat{P} = \mathcal{M}(\phi, U^{cpu}, B^{mem}) \tag{1}$$

Active initial profiling. In the absence of historical profiles (e.g., the initial stage), we employ an active probing mechanism to bootstrap the model $\mathcal{M}$. During the initial iterations of the analytics workload, we proactively execute a small set of decoding tasks (e.g., 30 samples, evaluated in §VI-C) as micro-probes across each node to serve as initial samples to fit $\mathcal{M}$ before repair occurs. These probes are configured with the same chunk size and RS parameters as the storage cluster. They are randomly scheduled within the iteration cycle to capture the decoding performance variability caused by the foreground workloads. The mechanism exploits the property that the computational behavior of analytics workloads is structurally deterministic, allowing us to fit the performance model from small-scale micro-probes [47]. Since distributed analytics workloads typically span multiple iterations, the probing cost is amortized across the entire execution, rendering its impact on foreground job performance negligible. When the workload pattern changes significantly, the model can be re-profiled using the same probing mechanism.

C. Modeling the Repair Problem

We model the repair problem following ChameleonEC [2].

Assumptions. Consider a stripe of $k+m$ chunks encoded with RS(k,m) over GF(2^ω), denoted as $\{D_1, D_2, \cdots, D_{k+m}\}$. Each chunk has size S. When node R requests a failed chunk D_f, a repair operation is triggered. Our objective is to minimize the total repair time (denoted as T_{total}) by strategically allocating repair tasks among the surviving nodes based on their computational and network capacities. The repair process involves two kinds of nodes as follows.

- **Sender node:** Each sender node i sends the fraction (η_i) of its local chunk to other nodes to help their repair tasks. The volume sent by node i to other nodes is $S \times \eta_i$.
- **Collector node:** Each collector node i collects its own fraction (ρ_i) of its local chunk (as a sub-chunk) as well as sub-chunks from other sender nodes, performs decoding operations, and uploads the reconstructed sub-chunk to R. The volume of the local sub-chunk is $\rho_i \times S$. The volume of sub-chunks collected from other nodes is $(k-1) \times S \times \rho_i$.

Repair time model. We characterize the computational and network capacities of node i with three metrics: decoding throughput (P_i), upload bandwidth (BW_i^{up}), and download bandwidth (BW_i^{down}). We obtain P_i with the method described in §IV-B, while BW_i^{up} and BW_i^{down} are measured directly with the `psutil` [32] module in Python. Since the repair process executes in a pipelined manner encompassing upload, download, and computation operations [2], [24], the repair operation time of node i, denoted as T_i, is determined by the bottleneck stage among the following three components:

- **Decoding time (T_i^{decode}):** The time required to perform decoding operations on all collected sub-chunks. The volume

processed is thus $k \times S \times \rho_i$.

$$T_i^{decode} = \frac{k \times \rho_i \times S}{P_i} \tag{2}$$

- **Upload time (T_i^{up}):** The time required to send sub-chunks to peers and transmit the reconstructed sub-chunk to R. The volume uploaded is thus $(\eta_i + \rho_i) \times S$.

$$T_i^{up} = \frac{(\eta_i+\rho_i)\times S}{BW_i^{up}} \tag{3}$$

- **Download time (T_i^{down}):** The time to download sub-chunks from other nodes to perform the decoding operations with its local sub-chunk so as to restore sub-chunk of D_f.

$$T_i^{down} = \frac{\rho_i \times (k-1) \times S}{BW_i^{down}}. \tag{4}$$

Thus, we can obtain T_i as

$$T_i = \max\left\{T_i^{decode}, T_i^{up}, T_i^{down}\right\}, \tag{5}$$

Finally, the whole repair process completes when R receives all reconstructed sub-chunks. Therefore, the total repair time T_{total} is determined by the slowest node. We formulate T_{total} as: $T_{total} = \max_{i \neq f}\{T_i\}$. Our goal is to determine ρ and η with given upload, download bandwidth and decoding throughput of all surviving nodes to minimize T_{total}, i.e.,

$$\min\{\max_{i \neq f}\{T_i\}\}, \tag{6}$$

which will be achieved in §V.

V. COAR Design

In this section, we introduce COAR, a computation-aware erasure-coded repair mechanism designed for distributed data analytics clusters with the following design goals.

- **Minimizing single-chunk repair time.** For single-chunk repair, COAR executes repair by jointly considering computational and network resources under distributed data analytics workloads (§V-A).
- **Accelerating multi-chunk repair.** For multi-chunk repair in a stripe, COAR reduces redundant network traffic and avoids redundant loading of multi-chunk repair context by selectively assigning tasks to the same nodes while preventing excessive computation load on any single node (§V-B).
- **Preventing stragglers in full-node repair.** For full-node repair, COAR proactively migrates repair tasks that are poised to run on potentially compute-busy nodes to compute-idle nodes, by leveraging the computation pattern of analytics jobs, thereby avoiding stragglers (§V-C).

A. COAR for single-chunk repair

To minimize single-chunk repair time under distributed data analytics workloads, we need to allocate repair tasks according to the optimization objective in Equation (6) (§IV-C). To this end, we first use a Linear Programming (LP) model to determine the proportion of repair tasks among the surviving nodes, based on their upload bandwidth, download bandwidth, and decoding throughput. Then we convert the solved task allocation into an executable repair plan.

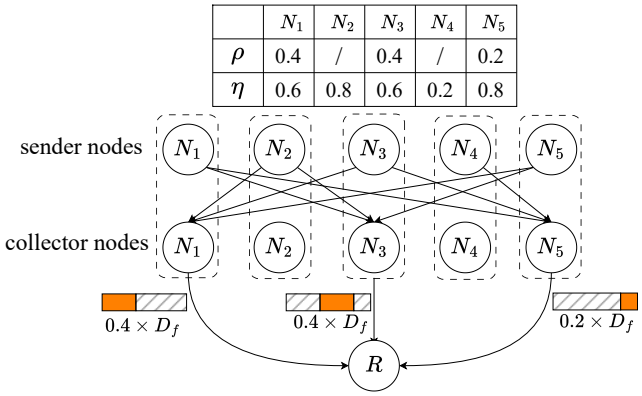


Figure 6: An example of the repair plan establishment. The top table shows the collect fraction (ρ) and send fraction (η) of each node. The bottom diagrams depict the generated data flows.

Allocating repair tasks with an LP solver. As formulated in §IV-B and §IV-C, the repair time is constrained by the maximum execution time among all nodes. Since the objective function (Equation (6)) and variables (ρ and η) of the repair time model can be formulated in linear form, we adopt an LP solver to derive the optimal ρ and η. The optimization problem is subject to the following three constraints.

- *Total Repair Constraint (**C1**).* The sum of all reconstructed sub-chunks must equal the full lost chunk D_f. Therefore, the sum of ρ is 1.

$$\sum_{i=1}^{k+m-1} \rho_i = 1 \tag{7}$$

- *Total Send Constraint (**C2**).* Repairing one chunk requires k chunks from the same stripe in total (§II-B). Thus, the total traffic provided from sender nodes to others is $k-1$ chunks (excluding sub-chunks read locally).

$$\sum_{i=1}^{k+m-1} \eta_i = k-1 \tag{8}$$

- *Stripe Constraint (**C3**).* While a node can serve as both sender and collector, the sub-chunks that it reconstructs for R and uploads to peers cannot exceed one chunk, since a node stores at most one chunk in a stripe for reliability (§II-B).

$$\eta_i + \rho_i \leq 1, \text{ where } 0 \leq \rho_i \leq 1, 0 \leq \eta_i \leq 1, 1 \leq i \leq k+m-1. \tag{9}$$

With the variables (ρ and η) and constraints ($C1$ to $C3$) in place, the LP solver numerically optimizes the objective function to derive the optimal ρ and η that minimize T_{total}.

Establishing the repair plan. With the task allocation (ρ and η) yielded, COAR materializes it to establish an executable repair plan based on the maximum flow model and Dinic's algorithm [7], similar to RepairBoost [25].

(Step 1): We model the data flow as a maximum flow problem. Specifically, we construct a flow network including: (i) a source N_S and a sink N_T; (ii) each node i serves as two logical entities: a sender node N_i^S and a collector node N_i^C; (iii) an edge from

N_S to N_i^C with capacity $(k-1)\times S\times \rho_i$, representing the total data volume that collector node i collects from other sender nodes; (iv) an edge from N_i^S to N_T with capacity $S\times \eta_i$, limiting the upload traffic of sender node i; (v) an edge from N_j^C to N_i^S with capacity $S\times \rho_j$, ensuring that collector node j retrieves no more than its local sub-chunk size from any sender node i. *(Step 2):* We use Dinic's algorithm to solve the above flow distribution among edges. The flow value on each edge (N_j^C, N_i^S) determines the sub-chunk size that node i transmits to node j.

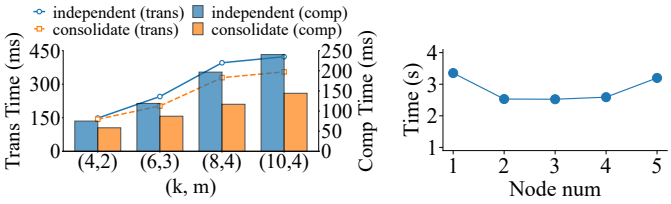
Fig. 6 illustrates an example of an RS(4,2) repair plan with five nodes (N_1 to N_5) to reconstruct chunk D_f. N_1, N_3, and N_5 serve as collector nodes with $\rho_1 = 0.4$, $\rho_3 = 0.4$, and $\rho_5 = 0.2$. COAR constructs a flow network to determine the sub-chunk transmission paths. Collector nodes N_1, N_3, N_5 reconstruct fractions of sizes $0.4\times D_f$, $0.4\times D_f$, and $0.2\times D_f$ respectively, which are finally aggregated to restore D_f.

Complexity analysis. Let $n = k+m-1$ denote the number of nodes. The LP model involves variables and linear constraints proportional to n. Solving it via Karmarkar's algorithm [17] yields a complexity of $O(n^{3.5})$. We construct a flow network comprising $V = 2n+2$ vertices and $E = 2n+n(n-1)$ edges. Dinic's algorithm incurs a complexity of $O(V^2E)$, which simplifies to $O(n^4)$. We evaluate the introduced computation overhead in §VI-C, confirming that it remains low.

B. COAR for multi-chunk repair within a stripe

Inefficiency of independent repair. Multi-chunk repair in a stripe involves reconstructing each failed chunk. A straightforward method is to apply COAR (§V-A) to each failure separately. However, such independent execution introduces two redundancies. First, each collector independently collects chunks from senders, making repair traffic f times for f failures compared to single-chunk repair. Second, each collector independently loads chunks and auxiliary data [22] from memory to the CPU, resulting in redundant loading, since both the chunks and auxiliary data (we refer to as the *multi-chunk repair context* for short) are identical across multiple repair tasks in a stripe and can be shared. To quantify the overhead incurred by independent execution, we measure the computation and transmission time of multi-chunk repair and compare independent execution (collect and load multi-chunk repair context for each task independently) with consolidated execution (collect and load multi-chunk repair context in a single collector). As shown in Fig. 7(a), independent execution yields higher transmission and computation time. For instance, with RS(10,4), the redundant collecting and loading incur a 15.99% penalty in transmission time and a 40.01% penalty in computation time compared to consolidated execution. These penalties demonstrate the overhead stemming from redundant multi-chunk repair context collecting and loading.

Insight to reuse multi-chunk repair context. Prior research [22] proposes that retaining auxiliary data of the first encoding operation in cache can accelerate subsequent encoding by reusing auxiliary data. This finding is also applicable to the multi-chunk repair context. To this end, assigning multiple



(a) Impact on transmission and computation. **(b)** Insight to selective reuse.

Figure 7: Impact of collecting and loading multi-chunk repair context. Note that in (a), we only measure the Galois Field computation and transmission time without foreground workloads to quantify the impact of collecting and loading multi-chunk repair context, excluding other operations overheads, such as memory allocation.

repair tasks to a single node allows the collector to reuse multi-chunk repair context retained in cache. It accelerates the repair process by eliminating redundant network traffic and avoiding redundant loading via multi-chunk repair context reuse.

However, performing reuse on repairing all failed chunks is not always beneficial. It means that if we assign all decoding operations to a single collector to make all failed chunk repairs share the multi-chunk repair context (i.e., maximizing reuse), then this node has excessive computational overhead, which is exacerbated under compute-intensive analytics workloads. Thus, we need to determine the number of collectors performing multi-chunk repair. To this end, we measure the total repair time of RS(10,5) for five failed chunks under the K-means clustering. We vary the number of collectors in multi-chunk repair from 1 (fully consolidated for maximum reuse) to 5 (fully distributed for no reuse). As shown in Fig. 7(b), the optimal performance is achieved with three collectors. It demonstrates the necessity of selective reuse to effectively accelerate multi-chunk repair.

Selective reuse for decoding operations. For multi-chunk repair in a stripe, we introduce COAR-SR, which extends COAR by selectively assigning tasks to nodes that already have repair tasks of the stripe. To quantify the benefit of reuse on decoding operations, we model the accelerated decoding throughput (denoted as P^{reuse}) based on the baseline throughput (denoted as P, obtained from Equation (1)). We calculate it as $P^{reuse} = \alpha \times P$, where α is a speedup factor representing the performance gain from reuse and is profiled offline.

We take the following three steps to achieve COAR-SR. *(Step 1):* Establish a repair plan for the first chunk following COAR (§V-A). Suppose that a node set $\mathbb{W}$ is selected to perform decoding operations. Let $\overline{\mathbb{W}}$ denote the set of nodes that are not selected in $\mathbb{W}$, and c be the node in $\overline{\mathbb{W}}$ which has the maximum repair capacity, defined as $\max_{c\in\overline{\mathbb{W}}}\left(\min\left\{\frac{P_c}{k}, \frac{BW_c^{down}}{k-1}\right\}\right)$. *(Step 2):* For subsequent failed chunks, we prioritize assigning decoding operations to node $i \in \mathbb{W}$ if its estimated processing time with reuse is faster than that when invoking node c:

$$\frac{k\times(L+1)}{P_i^{reuse}} < \min\{\frac{k}{P_c}, \frac{k-1}{BW_c^{down}}\} \tag{10}$$

Specifically, L represents the number of decoding operations with reuse that are already assigned to node i. The condition ensures that reuse is performed only when its benefit outweighs the advantage of utilizing a fresh node.

(Step 3): If no sufficient nodes in $\mathbb{W}$ satisfy the condition, we return to COAR and assign the remaining repair tasks to $\overline{\mathbb{W}}$.

C. COAR for full-node repair

Stragglers in full-node repair. Full-node repair involves restoring multiple chunks from different stripes. As a result, a node may have to process multiple independent repair tasks concurrently, making it a straggler. The straggler effect becomes more severe under compute-intensive analytics workloads.

Identifying computational states leveraging the staged computation pattern. As detailed in §IV-A, nodes in analytics clusters exhibit a staged computation pattern, alternating between compute-intensive and compute-idle stages with stable durations (see Fig. 5). This pattern allows us to predict computational state transitions and proactively migrate repair tasks from nodes in compute-intensive stages (i.e., stragglers) to nodes in compute-idle stages, thereby preventing performance degradation. To identify the upcoming compute-intensive stages, we maintain the durations of recent idle stages in historical profiles and track the duration of the current idle stage. We determine which node will transition to compute-intensive stages by predicting the idle stage duration via historical profiling.

Migrating tasks proactively. We extend COAR to COAR-MS. It migrates repair tasks from stragglers to nodes in compute-idle stages when a node is identified as transitioning to compute-intensive stages, with the following steps.

(Step 1): Select repair tasks on the straggler node, denoted as a set $\mathbb{T}_s$.

(Step 2): Select candidate migration destination nodes, denoted as $\mathbb{W}_{idle}$, which are currently in compute-idle stages. We sort $\mathbb{W}_{idle}$ in descending order of the repair capacity, calculated as $\min\left\{\frac{P_i}{k}, \frac{BW_i^{down}}{k-1}\right\}$ for node i.

(Step 3): Assign migration destination nodes from $\mathbb{W}_{idle}$ for tasks in $\mathbb{T}_s$ in descending order of their sub-chunk size. Specifically, we prioritize candidate nodes that host sub-chunk repair tasks belonging to the same chunk as the migrated tasks, because they have sub-chunks that can be reused without extra data transmission. If no such node exists, we select nodes from $\mathbb{W}_{idle}$ in a round-robin manner following the sorted order to balance load.

(Step 4): Migrate $\mathbb{T}_s$ to destination nodes. If necessary, it migrates the associated sub-chunks for decoding operations.

D. Discussion

Extension to other codes. COAR can be extended beyond RS codes. For locally repairable codes (LRCs) [14], which repair failures within a small local group of size r ($r < k$), COAR can adapt its repair time model (§IV-C) and constraints (§V-A) accordingly. For regenerating codes (RCs) [6], [10], which minimize repair bandwidth by requiring helpers to transmit encoded sub-blocks or linear combinations instead of full blocks, COAR can incorporate the encoding latency into its repair time model and schedule encoding tasks on computation-idle nodes to avoid shifting the bottleneck to encoding computation.

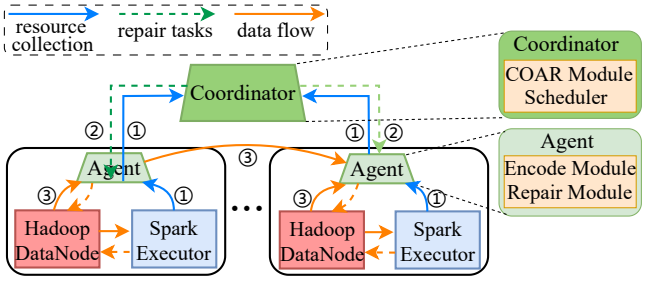


Figure 8: System architecture of COAR.

Extension to other analytics workloads. COAR relies on the periodic and staged computation patterns of distributed analytics workloads. However, some workloads may not have such patterns, such as single-stage computations, interactive analytics, and jobs triggered by human actions or AI-agent decisions. In such cases, COAR can still estimate decoding throughput from real-time CPU utilization and memory bandwidth usage, but its ability to predict future state transitions may be weakened. Therefore, COAR falls back to conservative repair scheduling based on online resource measurements and re-profiles the workload through active micro-probes when workload behavior deviates from historical profiles.

VI. Evaluation

A. Implementation

System architecture. We implement a COAR prototype in C++ and Python with about 6K SLoC, integrated with Apache Spark and Hadoop. As shown in Fig. 8, COAR consists of a centralized *Coordinator* and multiple distributed *Agents*. The coordinator runs on the master node and maintains global metadata, including stripe-to-chunk mappings and chunk placement; in production deployment, the metadata can be persisted in a replicated metadata service (e.g., ZooKeeper [15]). Upon triggering a repair, COAR generates a repair plan based on algorithms in §V. The scheduler dispatches repair tasks to agents. Agents are co-located with Spark executors and Hadoop DataNodes on worker nodes. They execute data analytics jobs and repair tasks, and report computational and network resources to the coordinator. They use the Repair Module for repair tasks and the Encode Module for coding operations. Erasure coding functionalities are realized via Jerasure v2.0 [16]. We also implement CR, PPR [26], RP [24], and ChameleonEC [2] (§II-B) for evaluation.

Repair Flow. Upon a chunk-loss report, the coordinator identifies the stripe information (e.g., chunk location, EC parameters), collects node upload, download bandwidth, and decoding throughput from agents (Step ①), generates a repair plan based on the computational and network resources of each agent, and dispatches tasks to agents (Step ②). The agents execute chunk transmission and decoding operations (Step ③). The coordinator updates stripe metadata after repair completes.

B. Experimental Setup

Testbed. We conduct experiments on a distributed cluster with 64 `ecs.c8i.xlarge` instances on Alibaba Cloud. Each

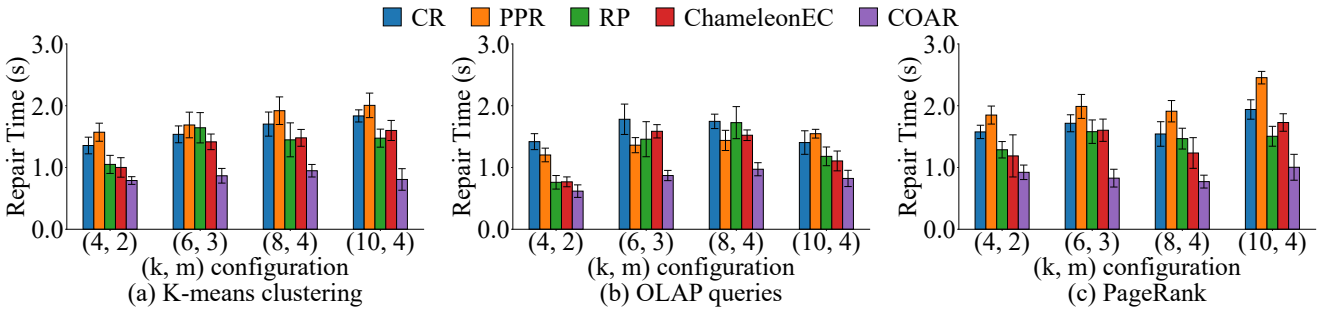


Figure 9: Total repair time of single-chunk (Exp. 1).

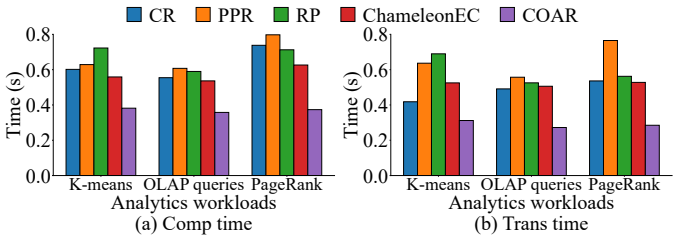


Figure 10: Decoding computation and transmission time of single-chunk repair (Exp. 2).

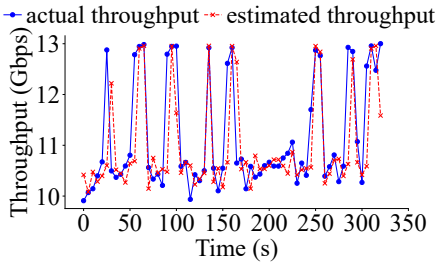


Figure 11: Estimation of decoding throughput (Exp. 3).

node is configured with an internal bandwidth of 15 Gbps. The cluster runs Hadoop 3.0.0 [11] as the underlying distributed file system, configured with a default chunk size of 64 MB. The default slice size is 1 MB. Apache Spark 2.4.0 [41] is deployed as the distributed data analytics framework.

Workloads. We employ the HiBench benchmark suite [12], which is widely adopted for characterizing distributed data analytics systems [28], [36], to generate datasets and run analytics workloads. To cover the diverse paradigms of distributed data analytics, we select three representative workloads, including K-means clustering, OLAP queries, and PageRank (§II-A), and run each workload in the foreground while triggering repair.

Comparative approaches. We compare COAR with CR, PPR [26], RP [24] and ChameleonEC [2]. We use RS (k,m) settings widely adopted in production systems, including (4,2) (a typical RAID-6 setting), (6,3) (QFS [30]), (8,4) (Baidu Atlas [18]), (10,4) (Facebook f4 [27]). To further study the impact of coding parameters, we extend experiments to include a broader range of (k,m). We plot average results of each experiment over 10 runs, with error bars representing the 95% confidence intervals calculated by Student's t-distribution.

C. Experimental Results

Exp. 1 (Total repair time of single-chunk): We evaluate the total repair time for single-chunk under three types of workloads and different RS configurations as shown in Fig. 9. COAR consistently outperforms other repair strategies. For example, under K-means clustering with RS(10,4) (Fig. 9(a)), COAR reduces the total repair time by 59.80% and 49.59% compared to PPR and ChameleonEC, respectively. While existing schemes optimize only for network bandwidth, they are unaware of computational states, often scheduling repair tasks to compute-intensive nodes saturated by foreground analytics workloads. In contrast, COAR strategically assigns tasks to compute-idle nodes by considering both computational and network resources, thereby effectively utilizing the computational resources.

Exp. 2 (Computation and transmission time): To investigate the source of the improvement provided by COAR, we compare COAR with other schemes by evaluating the decoding computation and transmission time of RS(6,3). Note that the results of pipelined schemes (e.g., RP, ChameleonEC, COAR) are averaged over participating nodes for transmission and computation. As shown in Fig. 10, COAR reduces both decoding computation and network transmission time. For instance, COAR reduces the computation and transmission time by 40.37% and 46.02% compared to ChameleonEC under PageRank. This is because COAR assigns repair tasks by jointly considering computational and network resources across nodes.

Exp. 3 (Effectiveness of periodicity-based estimation): We evaluate the effectiveness of the decoding throughput estimation. We first sample 30 data points across different iterations under K-means clustering for fitting. Then, we trigger decoding operations to collect actual throughputs and compare them against the predicted throughputs. As shown in Fig. 11, the results demonstrate that the estimation closely tracks the actual throughput dynamics, effectively capturing the peak loads and idle states. Quantitatively, the estimation achieves a Mean Absolute Percentage Error (MAPE) of 4.42%, ensuring that COAR operates with a precise view of computational capacities.

Exp. 4 (Scheduling overhead analysis): We evaluate the introduced scheduling overhead (§IV-B and §V-A), including decoding throughput prediction (XGBoost inference), task allocation (LP) and flow scheduling (Max Flow). As presented in Table I, the aggregate overhead accounts for 2.76%–4.77% of the total time, verifying that the introduced cost is low

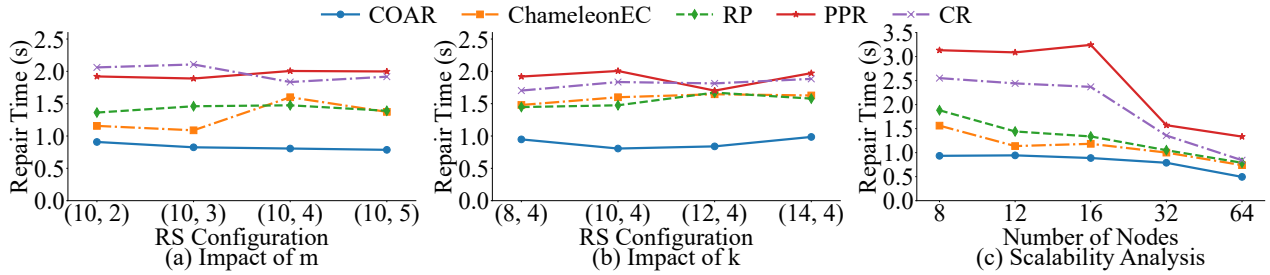


Figure 12: Impact of m, k and cluster scale (Exp. 5-7).

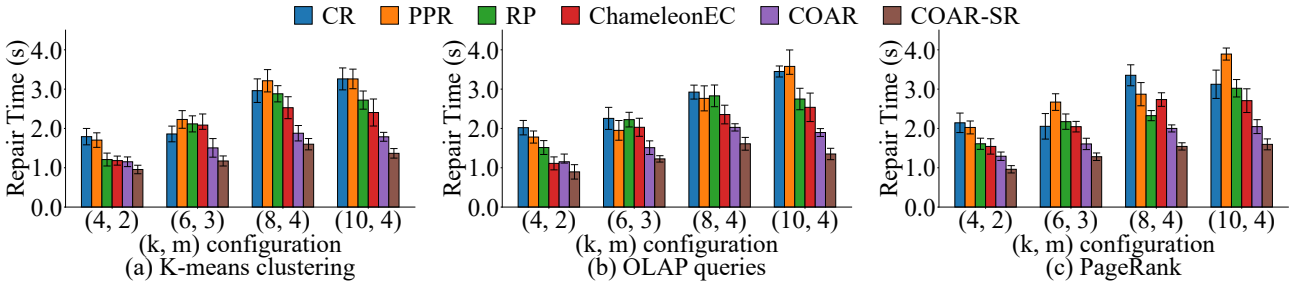


Figure 13: Total repair time of multi-chunk in a stripe (Exp. 8).

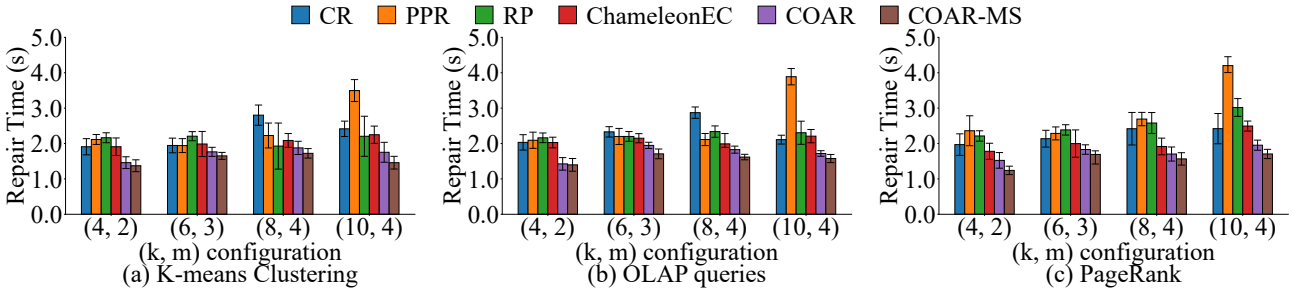


Figure 14: Total repair time of full-node (Exp. 9).

Table I: Overhead Analysis (Exp. 4).

(k,m)	XGBoost (ms)	LP (ms)	Max Flow (ms)	Ratio (%)
(4, 2)	19.69	5.58	0.08	2.76
(6, 3)	32.17	6.20	0.18	3.88
(8, 4)	39.63	8.59	0.26	4.68
(10, 4)	40.39	6.97	0.30	4.77

compared to the whole repair process.

Exp. 5 (Impact of RS redundancy (m)): We evaluate the single-chunk repair time versus RS redundancy. We fix k at 10 and vary m from 2 to 5, with K-means clustering as the foreground workload. As shown in Fig. 12(a), COAR maintains its advantage over other schemes across all m, demonstrating that COAR remains effective regardless of the RS redundancy. We observe a slight reduction in repair time of COAR as m increases. For instance, the repair time of COAR for RS(10,5) decreases by 13.41% compared to RS(10,2). It is attributed to increased scheduling flexibility, as a higher redundancy level (m) provides more candidate nodes, facilitating the identification of compute-idle nodes for repair.

Exp. 6 (Impact of stripe length (k)): We evaluate the total repair time for single-chunk versus stripe length. We fix m at 4 and vary k from 8 to 14. Results are presented in Fig. 12(b). While increasing stripe length (k) increases decoding computational complexity, COAR maintains stable repair efficiency relative to other approaches, demonstrating that COAR effectively adapts to varying stripe lengths.

Exp. 7 (Scalability of COAR): We evaluate the single-chunk repair time for RS(4,2) in clusters with 8, 12, 16, 32, and 64 nodes, with the K-means clustering workload running in the foreground. As shown in Fig. 12(c), COAR maintains its performance advantage over other schemes as the cluster scale increases. Notably, the repair time for all schemes decreases as the cluster size grows. This is because the foreground workload is distributed across more nodes, resulting in a lower load per node and leaving more available resources for repair.

Exp. 8 (Total repair time of multi-chunk): We set the failed chunk number in a stripe as m and evaluate total repair time. As shown in Fig. 13, COAR-SR shows advantages in multi-chunk repair. For instance, with RS(10,4) under K-means clustering, COAR-SR reduces repair time by 43.17% and 58.07% compared to ChameleonEC and CR (Fig. 13(a)). Additionally, under the above configuration, COAR-SR reduces the multi-chunk repair time by 23.53% over COAR, due to selective reuse of multi-chunk repair context in cache.

Exp. 9 (Total repair time of full-node): We write a number

of stripes randomly across nodes, erase 4 chunks in a node to mimic a full-node failure, and repair all failed chunks. Fig. 14 shows that COAR-MS outperforms other schemes in most cases. For example, under OLAP queries with RS(10,4) (Fig. 14(b)), COAR-MS reduces repair time by 28.64% and 59.47% against ChameleonEC and PPR, respectively. Moreover, COAR-MS outperforms COAR due to its ability to prevent repair tasks from executing on compute-intensive nodes. For example, COAR-MS reduces repair time of RS(4,2) under PageRank by 18.56% compared to COAR (Fig. 14(c)).

VII. Conclusion

We propose COAR, a computation-aware erasure-coded repair scheme designed for storage clusters in distributed data analytics. Leveraging the computation patterns of distributed data analytics, COAR schedules repair tasks by jointly considering computational and network resources. Experiments demonstrate COAR's efficiency in single-chunk, multi-chunk, and full-node repair across diverse analytics workloads.

References

[1] Alibaba Cloud. https://www.aliyun.com.

[2] Y. Cai, S. Lin, Z. Shen, J. Yang, and J. Shu, "ChameleonEC: Exploiting tunability of erasure coding for low-interference repair," in *Proc. of IEEE HPCA*, 2025.

[3] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. of ACM SIGKDD*, 2016.

[4] J. Darrous, S. Ibrahim, and C. Perez, "Is it time to revisit erasure coding in data-intensive clusters?" in *Proc. of IEEE MASCOTS*, 2019.

[5] C. Delimitrou and C. Kozyrakis, "Paragon: QoS-aware scheduling for heterogeneous datacenters," *ACM SIGPLAN Notices*, vol. 48, no. 4, pp. 77–88, 2013.

[6] A. G. Dimakis, P. B. Godfrey, Y. Wu, M. J. Wainwright, and K. Ramchandran, "Network coding for distributed storage systems," *IEEE Trans. on Information Theory*, vol. 56, no. 9, pp. 4539–4551, 2010.

[7] E. A. Dinic, "Algorithm for solution of a problem of maximum flow in networks with power estimation," in *Soviet Math. Doklady*, vol. 11, 1970, pp. 1277–1280.

[8] A. Fikes. (2010) Storage architecture and challenges. http://cloud.google.com/files/storage_architecture_and_challenges.pdf.

[9] J. Fried, Z. Ruan, A. Ousterhout, and A. Belay, "Caladan: Mitigating interference at microsecond timescales," in *Proc. of USENIX OSDI*, 2020.

[10] C. Gan, Y. Hu, L. Zhao, X. Zhao, P. Gong, and D. Feng, "Revisiting network coding for warm blob storage," in *Proc. of USENIX FAST*, 2025.

[11] Hadoop 3.0.0. https://hadoop.apache.org/docs/r3.0.0/.

[12] HiBench. https://github.com/Intel-bigdata/HiBench.

[13] Y. Hu, L. Cheng, Q. Yao, P. P. C. Lee, W. Wang, and W. Chen, "Exploiting combined locality for Wide-Stripe erasure coding in distributed storage," in *Proc. of USENIX FAST*, 2021.

[14] C. Huang, H. Simitci, Y. Xu, A. Ogus, B. Calder, P. Gopalan, J. Li, and S. Yekhanin, "Erasure coding in Windows Azure Storage," in *Proc. of USENIX ATC*, 2012.

[15] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *Proc. of USENIX ATC*, 2010.

[16] Jerasure. https://jerasure.org/.

[17] N. Karmarkar, "A new polynomial-time algorithm for linear programming," in *Proc. of ACM STOC*, 1984.

[18] C. Lai, S. Jiang, L. Yang, S. Lin, G. Sun, Z. Hou, C. Cui, and J. Cong, "Atlas: Baidu's key-value storage system for cloud data," in *Proc. of IEEE MSST*, 2015.

[19] J. Li and B. Li, "Zebra: Demand-aware erasure coding for distributed storage systems," in *Proc. of IEEE/ACM IWQoS*, 2016.

[20] ——, "On data parallelism of erasure coding in distributed storage systems," in *Proc. of IEEE ICDCS*, 2017.

[21] ——, "Parallelism-aware locally repairable code for distributed storage systems," in *Proc. of IEEE ICDCS*, 2018.

[22] Q. Li, L. Xu, Y. Li, M. Lyu, W. Wang, P. Zuo, and Y. Xu, "Enabling efficient erasure coding in disaggregated memory systems," *IEEE Trans. on Parallel and Distributed Systems*, vol. 35, no. 1, pp. 154–168, 2023.

[23] R. Li, P. P. C. Lee, and Y. Hu, "Degraded-first scheduling for MapReduce in erasure-coded storage clusters," in *Proc. of IEEE/IFIP DSN*, 2014.

[24] X. Li, Z. Yang, J. Li, R. Li, P. P. C. Lee, Q. Huang, and Y. Hu, "Repair pipelining for erasure-coded storage: Algorithms and evaluation," *ACM Trans. on Storage*, vol. 17, no. 2, pp. 1–29, 2021.

[25] S. Lin, G. Gong, Z. Shen, P. P. C. Lee, and J. Shu, "Boosting full-node repair in erasure-coded storage," in *Proc. of USENIX ATC*, 2021.

[26] S. Mitra, R. Panta, M.-R. Ra, and S. Bagchi, "Partial-parallel-repair (PPR): A distributed technique for repairing erasure coded storage," in *Proc. of ACM EuroSys*, 2016.

[27] S. Muralidhar, W. Lloyd, S. Roy, C. Hill, E. Lin, W. Liu, S. Pan, S. Shankar, V. Sivakumar, L. Tang, and S. Kumar, "f4: Facebook's warm BLOB storage system," in *Proc. of USENIX OSDI*, 2014.

[28] K. Ousterhout, C. Canel, S. Ratnasamy, and S. Shenker, "Monotasks: Architecting for performance clarity in data analytics frameworks," in *Proc. of ACM SOSP*, 2017.

[29] K. Ousterhout, R. Rasti, S. Ratnasamy, S. Shenker, and B.-G. Chun, "Making sense of performance in data analytics frameworks," in *Proc. of USENIX NSDI*, 2015.

[30] M. Ovsiannikov, S. Rus, D. Reeves, P. Sutter, S. Rao, and J. Kelly, "The Quantcast file system," *Proc. of the VLDB Endowment*, vol. 6, no. 11, pp. 1092–1101, 2013.

[31] J. W. Park, A. Tumanov, A. Jiang, M. A. Kozuch, and G. R. Ganger, "3sigma: Distribution-based cluster scheduling for runtime uncertainty," in *Proc. of ACM EuroSys*, 2018.

[32] psutil. https://pypi.org/project/psutil/.

[33] P. S. Rao and G. Porter, "Is memory disaggregation feasible? a case study with Spark SQL," in *Proc. of ACM/IEEE ANCS*, 2016.

[34] K. V. Rashmi, N. B. Shah, D. Gu, H. Kuang, D. Borthakur, and K. Ramchandran, "A solution to the network challenges of data recovery in erasure-coded distributed storage systems: A study on the Facebook warehouse cluster," in *Proc. of USENIX HotStorage*, 2013.

[35] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," *Journal of the Society for Industrial and Applied Mathematics*, vol. 8, no. 2, pp. 300–304, 1960.

[36] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, "Big data analytics on Apache Spark," *International Journal of Data Science and Analytics*, vol. 1, no. 3, pp. 145–164, 2016.

[37] M. Shen, Y. Zhou, and C. Singh, "Magnet: Push-based shuffle service for large-scale data processing," *Proc. of the VLDB Endowment*, vol. 13, no. 12, pp. 3382–3395, 2020.

[38] Z. Shen, Y. Cai, K. Cheng, P. P. C. Lee, X. Li, Y. Hu, and J. Shu, "A survey of the past, present, and future of erasure coding for storage systems," *ACM Trans. on Storage*, vol. 21, no. 1, pp. 1–39, 2025.

[39] H. Shin, K. Lee, and H.-Y. Kwon, "A comparative experimental study of distributed storage engines for big spatial data processing using GeoSpark," *The Journal of Supercomputing*, vol. 78, no. 2, pp. 2556–2579, 2022.

[40] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in *Proc. of IEEE MSST*, 2010.

[41] Spark. https://spark.apache.org.

[42] Spark GraphX. https://spark.apache.org/graphx/.

[43] Spark MLlib. https://spark.apache.org/mllib/.

[44] Spark SQL. https://spark.apache.org/sql.

[45] A. Thusoo, Z. Shao, S. Anthony, D. Borthakur, N. Jain, J. S. Sarma, R. Murthy, and H. Liu, "Data warehousing and analytics infrastructure at Facebook," in *Proc. of ACM SIGMOD*, 2010.

[46] V. K. Vavilapalli, A. C. Murthy, C. Douglas, S. Agarwal, M. Konar, R. Evans, T. Graves, J. Lowe, H. Shah, S. Seth, B. Saha, C. Curino, O. O'Malley, S. Radia, B. Reed, and E. Baldeschwieler, "Apache Hadoop YARN: Yet another resource negotiator," in *Proc. of ACM SoCC*, 2013.

[47] S. Venkataraman, Z. Yang, M. Franklin, B. Recht, and I. Stoica, "Ernest: efficient performance prediction for large-scale advanced analytics," in *Proc. of USENIX NSDI*, 2016.

[48] X. Yao, C.-L. Wang, and M. Zhang, "Ec-shuffle: Dynamic erasure coding optimization for efficient and reliable shuffle in spark," in *Proc. of IEEE/ACM CCGrid*, 2019.

[49] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, "Resilient distributed datasets: a fault-tolerant abstraction for in-memory cluster computing," in *Proc. of USENIX NSDI*, 2012.